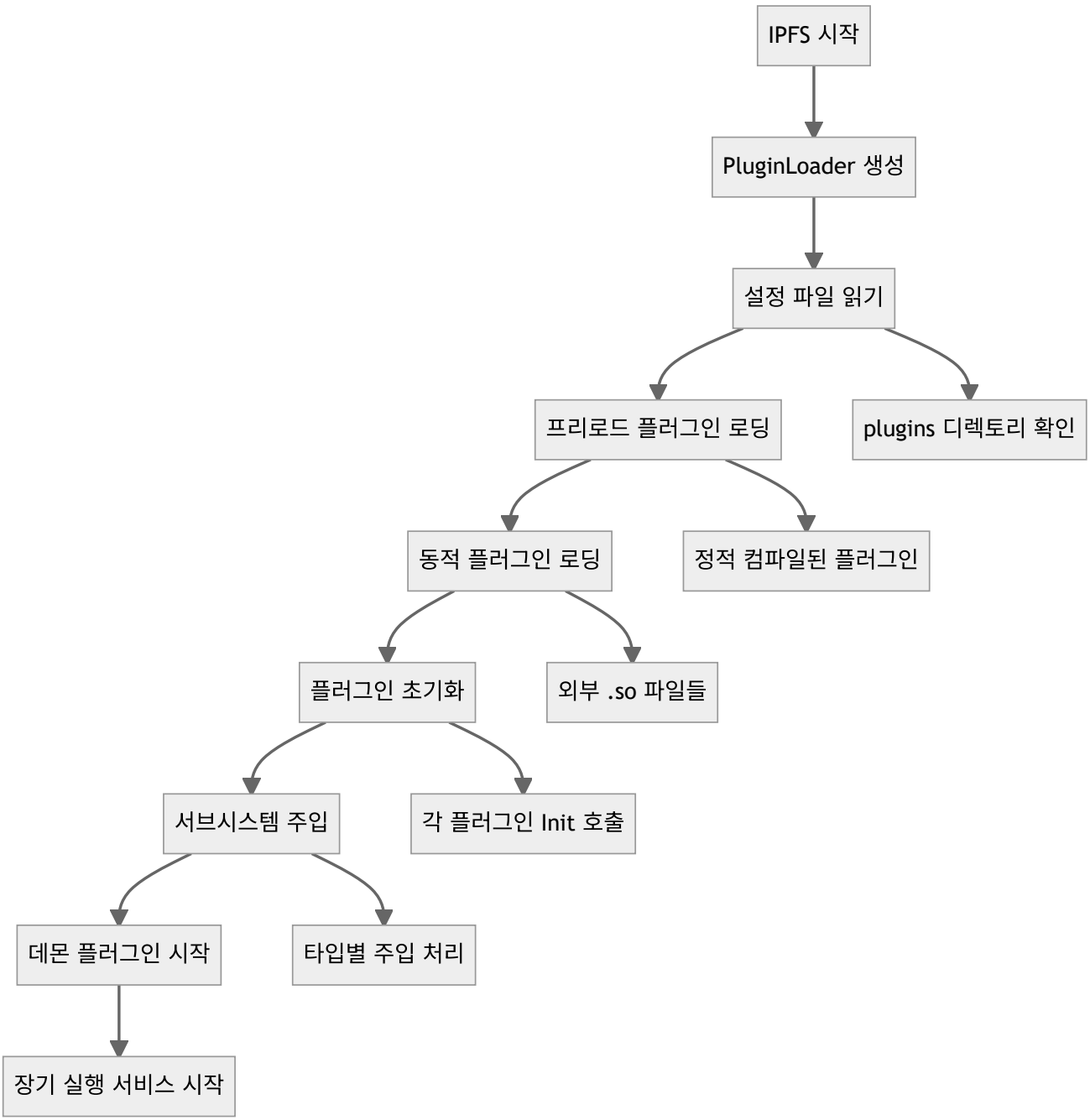
IPFS 플러그인 로딩 메커니즘 문서화

개요

IPFS Kubo의 플러그인 로딩 시스템은 정적 프리로드와 동적 로딩을 결합한 하이브리드 접근 방식을 사용합니다. 이 시스템은 플랫폼별 구현을 통해 다양한 환경에서의 호환성을 보장하며, 설정 기반의 플러그인 관리를 제공합니다.

플러그인 로딩 아키텍처

전체 로딩 프로세스



동적 플러그인 로딩 과정

1. 플랫폼별 로딩 구현

IPFS는 빌드 태그를 사용하여 플랫폼별로 다른 로딩 메커니즘을 제공합니다:

Unix 시스템 (CGO 지원)

```
//go:build cgo && !noplugin && (linux || darwin || freebsd)

func unixLoadPlugin(fi string) ([]iplugin.Plugin, error) {
// Go의 plugin 패키지를 사용한 동적 로딩
pl, err := plugin.Open(fi)
if err != nil {
return nil, err
}
// 플러그인에서 "Plugins" 심볼 찾기
pls, err := pl.Lookup("Plugins")
if err != nil {
return nil, err
}

// 타입 검증
typePls, ok := pls.(*[]iplugin.Plugin)
if !ok {
return nil, errors.New("filed 'Plugins' didn't contain correct type")
}

return *typePls, nil
```

```
}
```

특징:

- Go의 표준 `plugin` 패키지 사용
- 공유 라이브러리(.so) 파일 동적 로딩
- 런타임에 심볼 해석

CGO 미지원 환경

```
//go:build !cgo && !noplugin && (linux || darwin || freebsd)


func nocgoLoadPlugin(fi string) ([]iplugin.Plugin, error) {
return nil, errors.New("not built with cgo support")
}
```

특징:

- CGO 없이 빌드된 경우 동적 로딩 불가
- 정적 프리로드 플러그인만 사용 가능

플러그인 지원 비활성화

```
//go:build noplugin


func nopluginLoadPlugin(string) ([]iplugin.Plugin, error) {
return nil, errors.New("not built with plugin support")
```

```
}
```

특징:

- 보안상의 이유로 플러그인 지원 완전 비활성화
- 최소한의 기능만 제공

2. 동적 로딩 프로세스

```
func loadDynamicPlugins(pluginDir string) ([]plugin.Plugin, error) {
// 플러그인 디렉토리 존재 확인
_, err := os.Stat(pluginDir)
if os.IsNotExist(err) {
return nil, nil
}

var plugins []plugin.Plugin

// 디렉토리 순회
err = filepath.Walk(pluginDir, func(fi string, info os.FileInfo, err error) er
if err != nil {
return err
}
// 디렉토리는 스킵 (경고 출력)
if info.IsDir() {
```

```
if fi != pluginDir {
log.Warnf("found directory inside plugins directory: %s", fi)
}
return nil
}

// 보안 검사: 실행 권한 확인
if info.Mode().Perm()&0o111 == 0 {
log.Errorf("non-executable file in plugins directory: %s", fi)
return nil
}

// 플러그인 로딩 시도
if newPlugins, err := loadPluginFunc(fi); err == nil {
plugins = append(plugins, newPlugins...)
} else {
return fmt.Errorf("loading plugin %s: %s", fi, err)
}
return nil
})

return plugins, err
```

```
}
```

보안 고려사항:

1. **실행 권한 검사**: 실행 권한이 없는 파일은 로딩하지 않음
2. **디렉토리 검사**: 플러그인 디렉토리 내 하위 디렉토리 경고
3. **오류 전파**: 로딩 실패 시 명확한 오류 메시지 제공

정적 플러그인 프리로드 시스템

1. 프리로드 목록 관리

preload_list 파일 형식:

```
# 주석 라인 (# 으로 시작)
# name go-path number of the sub-plugin or *

ipldgit github.com/ipfs/kubo/plugin/plugins/git *
ipld dagjose github.com/ipfs/kubo/plugin/plugins/dagjose *
badgerds github.com/ipfs/kubo/plugin/plugins/badgerds *
flatfs github.com/ipfs/kubo/plugin/plugins/flatfs *
levelds github.com/ipfs/kubo/plugin/plugins/levelds *
pebbleds github.com/ipfs/kubo/plugin/plugins/pebbleds *
peerlog github.com/ipfs/kubo/plugin/plugins/peerlog *
```

필드 설명:

- **name**: 플러그인 식별자

- **go-path**: Go 패키지 경로
- **number**: 특정 플러그인 인덱스 또는 `*` (모든 플러그인)

2. 자동 코드 생성

preload.sh 스크립트 동작:

```
#!/usr/bin/env bash


# 프리로드할 플러그인 목록 생성
to_preload() {
awk 'NF' $LIST | sed '/^#/d' # 빈 줄과 주석 제거
if [[ -n "$IPFS_PLUGINS" ]]; then
# 환경 변수로 추가 플러그인 지정 가능
for plugin in $IPFS_PLUGINS; do
echo "$plugin github.com/ipfs/kubo/plugin/plugins/$plugin *"
done
fi
}


# Go 코드 생성
cat <<EOL
package loader


import (
```

```
EOL

# import 문 생성
to_preload | while read -r name path num; do
echo "plugin$name \"$path\""
done | sort -u

cat <<EOL
)

func init() {
EOL

# Preload 호출 생성
to_preload | while read -r name path num; do
case "$num" in
'*') echo " Preload(plugin$name.Plugins...)" ;;
*) echo " Preload(plugin$name.Plugins[$num])" ;;
esac
done

echo "}"
```

생성된 preload.go 예시:

```
package loader

import (
pluginbadgerds "github.com/ipfs/kubo/plugin/plugins/badgerds"
pluginiplddagjose "github.com/ipfs/kubo/plugin/plugins/dagjose"
pluginflatfs "github.com/ipfs/kubo/plugin/plugins/flatfs"
// ... 기타 플러그인들
)

func init() {
Preload(pluginipldgit.Plugins...)
Preload(pluginiplddagjose.Plugins...)
Preload(pluginbadgerds.Plugins...)
// ... 기타 플러그인들
}
```

3. 프리로드 메커니즘

```
var preloadPlugins []plugin.Plugin

func Preload(plugins ...plugin.Plugin) {
preloadPlugins = append(preloadPlugins, plugins...)
```

```
}


func NewPluginLoader(repo string) (*PluginLoader, error) {
loader := &PluginLoader{
plugins: make([]plugin.Plugin, 0, len(preloadPlugins)),
repo: repo,
}
// 프리로드된 플러그인들을 로더에 추가
for _, v := range preloadPlugins {
if err := loader.Load(v); err != nil {
return nil, err
}
}
return loader, nil
}
```

플러그인 설정 및 활성화/비활성화

1. 설정 구조

```
type Plugins struct {
Plugins map[string]Plugin
}
```

```
type Plugin struct {
Disabled bool // 플러그인 비활성화 여부
Config interface{} // 플러그인별 설정
}
```

2. 설정 파일 예시

```
{
"Plugins": {
"Plugins": {
"peerlog": {
"Disabled": false,
"Config": {
"Enabled": true,
"LogLevel": "info"
}
},
"badgerds": {
"Disabled": true,
"Config": {
"path": "badgerds",
"syncWrites": false,
```

```
"vlogFileSize": "1GiB"
}
}
}
}
}
```

3. 설정 로딩 과정

```
func readPluginsConfig(repoRoot string, userConfigFile string) (config.Plugin
var cfg struct {
Plugins config.Plugins
}

cfgPath, err := config.Filename(repoRoot, userConfigFile)
if err != nil {
return config.Plugins{}, err
}

cfgFile, err := os.Open(cfgPath)
if err != nil {
return config.Plugins{}, err
}
```

```
defer cfgFile.Close()

// JSON 디코딩으로 Plugins 섹션만 읽기
err = json.NewDecoder(cfgFile).Decode(&cfg)
if err != nil {
return config.Plugins{}, err
}

return cfg.Plugins, nil
}
```

특징:

- **부분 설정 로딩**: 전체 설정이 아닌 Plugins 섹션만 로딩
- **마이그레이션 안전성**: 다른 설정 섹션 변경에 영향받지 않음
- **오류 허용**: 설정 파일이 없어도 계속 진행

4. 플러그인 비활성화 처리

```
func (loader *PluginLoader) Load(pl plugin.Plugin) error {
name := pl.Name()

// 중복 플러그인 검사
for _, p := range loader.plugins {
```

```
if p.Name() == name {
return fmt.Errorf(
"plugin: %s, is duplicated in version: %s, "+
"while trying to load dynamically: %s",
name, p.Version(), pl.Version())
}
}


// 비활성화된 플러그인 스킵
if loader.config.Plugins[name].Disabled {
log.Infof("not loading disabled plugin %s", name)
return nil
}
loader.plugins = append(loader.plugins, pl)
return nil
}
```

플러그인 간 상호작용 및 충돌 해결

1. 이름 기반 충돌 해결

```
func (loader *PluginLoader) Load(pl plugin.Plugin) error {
name := pl.Name()
```

```
// 동일한 이름의 플러그인 검사
for _, p := range loader.plugins {
if p.Name() == name {
return fmt.Errorf(
"plugin: %s, is duplicated in version: %s, "+
"while trying to load dynamically: %s",
name, p.Version(), pl.Version())
}
}
return nil
}
```

충돌 해결 전략:

1. **이름 유일성**: 플러그인 이름은 전역적으로 유일해야 함
2. **버전 정보**: 충돌 시 두 플러그인의 버전 정보 제공
3. **로딩 순서**: 프리로드 플러그인이 동적 플러그인보다 우선

2. 타입별 상호작용 관리

```
func (loader *PluginLoader) Inject() error {
for _, pl := range loader.plugins {
// IPLD 플러그인 처리
if pl, ok := pl.(plugin.PluginIPLD); ok {
```

```
err := injectIPLDPlugin(pl)

if err != nil {

loader.state = loaderFailed

return err

}

}

// 데이터스토어 플러그인 처리

if pl, ok := pl.(plugin.PluginDatastore); ok {

err := injectDatastorePlugin(pl)

if err != nil {

loader.state = loaderFailed

return err

}

}

// 기타 플러그인 타입들...

}

return nil

}
```

상호작용 원칙:

1. **타입 격리**: 각 플러그인 타입은 독립적으로 처리
2. **순차 처리**: 플러그인들은 순차적으로 주입
3. **실패 전파**: 하나의 플러그인 주입 실패 시 전체 중단

3. 의존성 해결

```
func injectDatastorePlugin(pl plugin.PluginDatastore) error {
// 데이터스토어 설정 핸들러 등록
return fsrepo.AddDatastoreConfigHandler(
pl.DatastoreTypeName(),
pl.DatastoreConfigParser(),
)
}

func injectIPLDPlugin(pl plugin.PluginIPLD) error {
// IPLD 멀티코덱 레지스트리에 등록
return pl.Register(multicodec.DefaultRegistry)
}
```

의존성 관리:

1. **레지스트리 패턴**: 각 서브시스템은 자체 레지스트리 유지
2. **지연 바인딩**: 실제 사용 시점에 플러그인 해결
3. **오류 격리**: 개별 플러그인 오류가 다른 플러그인에 영향 없음

플러그인 초기화 과정

1. 환경 설정 전달

```
func (loader *PluginLoader) Initialize() error {
for _, p := range loader.plugins {
env := &plugin.Environment{
Repo: loader.repo,
Config: loader.config.Plugins[p.Name()].Config,
}
err := p.Init(env)
if err != nil {
loader.state = loaderFailed
return fmt.Errorf("initializing plugin %s: %w", p.Name(), err)
}
}
return nil
}
```

환경 정보:

- **Repo**: IPFS 저장소 경로
- **Config**: 플러그인별 설정 데이터

2. 플러그인별 초기화 예시

데이터스토어 플러그인:

```
func (*badgerdsPlugin) Init(_ *plugin.Environment) error {
// 대부분의 데이터스토어 플러그인은 초기화 로직이 단순
```

```
return nil
}
```

데몬 플러그인:

```
func (pl *peerLogPlugin) Init(env *plugin.Environment) error {
pl.events = make(chan plEvent, eventQueueSize)
pl.enabled = extractEnabled(env.Config)
return nil
}
```

플러그인 시작 및 생명주기 관리

1. 데몬 플러그인 시작

```
func (loader *PluginLoader) Start(node *core.IpfsNode) error {
iface, err := coreapi.NewCoreAPI(node)
if err != nil {
return err
}
for _, pl := range loader.plugins {
// 외부 API 접근 플러그인
if pl, ok := pl.(plugin.PluginDaemon); ok {
err := pl.Start(iface)
```

```
if err != nil {
_ = loader.Close()
return err
}
loader.started = append(loader.started, pl)
}
// 내부 API 접근 플러그인
if pl, ok := pl.(plugin.PluginDaemonInternal); ok {
err := pl.Start(node)
if err != nil {
_ = loader.Close()
return err
}
loader.started = append(loader.started, pl)
}
}
return nil
}
```

2. 플러그인 종료 처리

```
func (loader *PluginLoader) Close() error {
var errs []string
```

```
started := loader.started
loader.started = nil
for _, pl := range started {
if closer, ok := pl.(io.Closer); ok {
err := closer.Close()
if err != nil {
errs = append(errs, fmt.Sprintf(
"error closing plugin %s: %s",
pl.Name(),
err.Error(),
))
}
}
}
if errs != nil {
loader.state = loaderFailed
return errors.New(strings.Join(errs, "\n"))
}
loader.state = loaderClosed
return nil
}
```

생명주기 관리 특징:

1. **시작된 플러그인 추적**: 시작된 플러그인만 종료 처리
2. **오류 수집**: 모든 종료 오류를 수집하여 한 번에 반환
3. **상태 관리**: 로더 상태를 적절히 업데이트

고급 로딩 기능

1. 환경 변수를 통한 동적 플러그인 추가

```
# 빌드 시 추가 플러그인 지정
export IPFS_PLUGINS="customplugin anotherplugin"
make build
```

preload.sh에서의 처리:

```
if [[ -n "$IPFS_PLUGINS" ]]; then
for plugin in $IPFS_PLUGINS; do
echo "$plugin github.com/ipfs/kubo/plugin/plugins/$plugin *"
done
fi
```

2. 조건부 플러그인 로딩

```
func extractEnabled(config interface{}) bool {
if config == nil {
return false
```

```
}
mapIface, ok := config.(map[string]interface{})
if !ok {
return false
}
enabledIface, ok := mapIface["Enabled"]
if !ok || enabledIface == nil {
return false
}
enabled, ok := enabledIface.(bool)
if !ok {
return false
}
return  enabled
}
```

3. 플러그인 상태 모니터링

```
type loaderState int

const (
loaderLoading loaderState = iota
loaderInitializing
```

```
loaderInitialized

loaderInjecting

loaderInjected

loaderStarting

loaderStarted

loaderClosing

loaderClosed

loaderFailed

)


func (ls loaderState) String() string {

switch  ls {

case  loaderLoading:

return  "Loading"

case  loaderInitializing:

return  "Initializing"

// ... 기타 상태들

default:

return  "Unknown"

}

}
```

이 플러그인 로딩 메커니즘은 IPFS Kubo의 확장성과 유연성을 보장하면서도, 보안과 안정성을 유지하는 정교한 시스템입니다. 정적 프리로드와 동적 로딩의 조합을 통해 다양한 사용 사례와 배포 환경을 지원합니다.